

# Project 3 — Drone-Based Delivery Optimization

Dataset Documentation • data-v3.json

Team Notes • May 2026

## 1. Overview

---

This doc explains the structure of data-v3.json — the benchmark dataset we built for our drone routing problem. Read this before touching any code so we're all on the same page about what the data contains and how to use it.

This is data-v3.json, the final version. Key changes from v2: NFZ reduced to 10 meaningful zones, original depot restored (Algiers Port), all depot to customer edges guaranteed clear, INST\_04 now uses 5 drones.

**The file has three top-level keys:**

- `metadata` — global info about the dataset (version, energy model, drone spec, etc.)
- `city_template` — the shared Algiers map: depot location, 14 neighborhoods, and 10 no-fly zones
- `instances` — the list of 10 problem instances we will run our algorithms on

## 2. The 10 Benchmark Instances

---

We have 10 instances split across 4 size categories. The idea is to test our algorithms on problems ranging from trivially easy (8 customers) to genuinely hard (75 customers), so our experimental analysis is meaningful.

| Instance ID | Category | Customers | Drones | Drone Payload |
|-------------|----------|-----------|--------|---------------|
| INST_01     | Small    | 8         | 2      | 5.0 kg        |
| INST_02     | Small    | 12        | 3      | 5.0 kg        |
| INST_03     | Small    | 15        | 3      | 5.0 kg        |
| INST_04     | Medium   | 20        | 5      | 5.0 kg        |
| INST_05     | Medium   | 25        | 5      | 5.0 kg        |
| INST_06     | Medium   | 30        | 6      | 5.0 kg        |
| INST_07     | Large    | 40        | 8      | 5.0 kg        |
| INST_08     | Large    | 50        | 10     | 5.0 kg        |
| INST_09     | XLarge   | 60        | 12     | 5.0 kg        |
| INST_10     | XLarge   | 75        | 15     | 5.0 kg        |

All drones are identical (homogeneous fleet) — see Section 4 for the full drone spec. The number of drones scales roughly with problem size so that instances remain feasible but not trivially so.

### 3. Customer Data

---

Each customer in an instance looks like this:

```
{
  "customer_id": 3,
  "name": "Ouled_Fayet_C3",
  "neighborhood": "Ouled_Fayet",
  "latitude": 36.737199,
  "longitude": 2.943809,
  "demand_kg": 1.5
}
```

A few things to note:

- Coordinates are real GPS points inside the 14 Algiers neighborhoods we selected.
- demand\_kg is drawn from [0.25, 0.5, 0.75, 1.0, 1.5] kg.
- In our problem a drone can carry multiple packages per trip (multi-drop). The drone picks up all packages for its assigned customers at the depot, drops them one by one along the route, and returns. Payload decreases at each stop.

### 4. Drone Specification

---

All drones are identical across all instances:

| Parameter               | Value                   |
|-------------------------|-------------------------|
| Battery capacity        | 2.5 kWh                 |
| Max payload             | 5.0 kg                  |
| Max range               | 56 km                   |
| Base energy consumption | 44 Wh/km (empty drone)  |
| Load factor             | 8 Wh/km per kg of cargo |
| Cruise speed            | 50 km/h                 |

### 5. Energy Model

---

We use a load-dependent linear energy model. The energy consumed on a single leg is:

$$E = d \times (44 + 8 \times w) / 1000 \quad [\text{kWh}]$$

Where:

- $d$  = distance of the leg in km
- $w$  = payload being carried on that leg in kg
- 44 = base consumption in Wh/km (drone flying empty)
- 8 = extra Wh/km for every kg of cargo

**Important: compute energy leg by leg, not for the whole route at once.**

Since the drone drops packages at each stop, the payload  $w$  decreases along the route. Example for a route depot → C1 (1.5 kg) → C2 (1.0 kg) → depot:

```
Leg depot → C1: w = 2.5 kg → E = d01 × (44 + 8×2.5) / 1000
Leg C1 → C2: w = 1.0 kg → E = d12 × (44 + 8×1.0) / 1000
Leg C2 → depot: w = 0.0 kg → E = d20 × (44 + 8×0.0) / 1000
```

The total route energy is the sum of all legs. It must not exceed 2.5 kWh (battery capacity).

The data file includes precomputed energy matrices at payload levels 0, 1, 2, 3, 4, and 5 kg under `energy_matrices_kwh.by_payload_kg`. For intermediate payload values, interpolate linearly between the two closest levels.

## 6. Distance Matrix

Each instance contains a `distance_matrix_km`: an  $(N+1) \times (N+1)$  matrix where  $N$  is the number of customers. Node 0 is always the depot; nodes 1 to  $N$  are customers in order.

Distances are computed using the Haversine formula — the standard way to get straight-line distances from GPS coordinates. We use Haversine rather than Euclidean distance because the coordinates are real latitude/longitude values, and plain Euclidean distance on lat/lon gives wrong results (it ignores the curvature of the Earth).

To look up the distance between two nodes:

```
dist = instance['distance_matrix_km'][i][j]
```

The matrix is symmetric (`dist[i][j] == dist[j][i]`) and has zeros on the diagonal.

## 7. No-Fly Zones

We use 10 carefully selected no-fly zones sourced from OpenStreetMap for the Algiers area (reduced from the original 45 to ensure all instances remain feasible):

| Category    | Count | Example                                                     |
|-------------|-------|-------------------------------------------------------------|
| Aerodrome   | 2     | Houari Boumediene Airport, Chéraga Military Airport         |
| Military    | 6     | Commandement des Forces Terrestres, Garde Républicaine, ... |
| Power Plant | 2     | Centrale électrique d'El Hamma, Centrale de Bab Ezzouar     |

Each zone is defined by a bounding box (min\_lat, max\_lat, min\_lon, max\_lon). A drone cannot fly through any of these zones.

### How `nfz_blocked_edges` works

To avoid doing geometry checks inside your solver at every step, each instance precomputes which edges are blocked. The `nfz_blocked_edges` field is a dictionary:

```
"nfz_blocked_edges": {
  "edges": {
    "0-3": ["Aéroport international d'Alger"],
    "2-7": ["Commandement des Forces Terrestres"],
    ...
  }
}
```

The key "0-3" means the edge between node 0 (depot) and node 3 (customer 3) crosses at least one no-fly zone. The value is the list of NFZ names that block it.

To check if a leg is feasible in your code:

```
i, j = min(a, b), max(a, b)  # always put smaller index first
blocked = f'{i}-{j}' in instance['nfz_blocked_edges']['edges']
```

Use this check:

- In Branch & Bound — prune partial routes that include a blocked edge
- In Genetic Algorithm — your repair operator must replace or reroute blocked legs
- In local search (SA/Tabu) — reject any neighbor move that introduces a blocked edge

## 8. Quick Reference

---

`node_labels` is a convenience list so you can print readable route names:

```
node_labels = ['depot', 'C1', 'C2', ..., 'CN']  
# e.g. route [0, 3, 7, 0] prints as: depot → C3 → C7 → depot
```

A valid route for one drone must:

- Start and end at node 0 (depot)
- Visit each assigned customer exactly once
- Not exceed 2.5 kWh total energy (computed leg by leg with load-dependent formula)
- Not exceed 5.0 kg total payload at departure
- Not use any edge listed in `nfz_blocked_edges`